Understanding Objects in JavaScript

Objects are the fundamental building blocks of JavaScript, allowing you to organize and structure data in powerful ways. They store complex collections of related information and functionality, making your code more organized and maintainable.

Data Storage

Group related complex values together using key-value pairs

Flexibility

Objects can hold any data type including other objects

Real-World Modeling

Represent real entities like users, products, or settings

Objects are collections of properties, where each property is a (key, value) pair used to store keyed collections and complex entities.

Note: We typically define objects using the const keyword to prevent reassignment.

```
const student = {
  name: "nithin",
  age: 23,
  marks: 94.4
};

const item = {
  price: 100.99,
  discount: 50,
  colors: ["red", "pink"]
};
```

Practice Question

Create an object for the properties of twitter post which includes -

- username (post)
- content
- likes
- reposts
- tags

```
const post = {
    username: "@Nithin",
    content: "This is my #firstPost",
    likes: 150,
    reposts: 5,
    tags: ["opqtech", "fsd_2026"]
};
```

Get Values

To access values from JavaScript objects, you first define the object with its properties:

```
let student = {
  name: "nithin",
  marks: 94.4
};
```

There are two primary ways to access the values associated with these properties:

```
student["name"]
```

```
student.name
```

The first method, **bracket notation** (`student["name"]`), uses square brackets and a string literal for the property name. The second, **dot notation** (`student.name`), uses a dot followed by the property name. Both methods achieve the same result of retrieving the value associated with the 'name' property.

> JS automatically converts objects keys to strings.
>
> Even if we made the number as a key, the number will be converted to string.

Add/ Update Value

Tasks:

- Change the city to "Mumbai"
- Add a new property, gender: "Female"
- Change the marks to "A"

Example:

```
const student = {
  name: "Nithin",
  age: 23,
  marks: 94.4,
  city: "Delhi"
};
```

To delete the `city` property from the `student` object:

```
delete student.city;
```

The `delete` operator removes a property from an object. After deletion, the property no longer exists in the object.

Nested Objects

Objects in JavaScript can contain other objects as property values, allowing you to create complex and hierarchical data structures. This is known as "nested objects".

```
const student = {
  name: "Nithin",
  age: 23,
  marks: {
    math: 95,
    science: 88,
    english: 92
  },
  address: {
    city: "Delhi",
    country: "India"
  }
};
```

You can access nested properties using dot notation:

```
student.marks.math  // 95
student.address.city  // "Delhi"
```

Alternatively, you can use bracket notation:

```
student["marks"]["math"]  // 95
```

Objects can be nested as deeply as needed to accurately represent complex data structures, providing a flexible way to organize related data.

Array of Objects

Arrays in JavaScript can store multiple objects as their elements. This powerful feature is known as an "array of objects" and is incredibly useful for managing collections of similar items, such as a list of students, products, or users.

```
const students = [
  {
    name: "nithin",
    age: 23,
    marks: 94.4
  },
  {
    name: "rahul",
    age: 22,
    marks: 88.5
  },
  {
    name: "priya",
    age: 24,
    marks: 91.2
  }
];
```

You can access individual objects in the array using their index, and then access their properties using dot notation:

```
students[0]  // First student object
students[0].name  // "nithin"
students[1].marks  // 88.5
```

This pattern is extremely common and fundamental in real-world applications for effectively managing and iterating through lists of structured data.

Math Object

Properties

Math.PI

Math.E

Methods

Math.abs(n)

Math.pow(a, b)

Math.floor(n)

Math.ceil(n)

Math.random( )

Random Integers

From 1 to 10

Step1 :

```
let num = Math.random();
```

```
0.46747741318
```

Step2 :

```
num = num * 10;
4.674774131812704
```

Step3 :

```
num = Math.floor(num);
4
```

Step4 :

```
num = num + 1;
5
```

Qs

Generate a random number between 1 and 100.

Math.random()

Generate a random number between 1 and 5.

Guessing Game

User enters a max number & then tries to guess a random generated number between 1 to max.